

REST API Mini Project – Employee Management System

Name: Pakala Neha

Objective

To configure JSON Server, create a Fake REST API using db.json, and test all REST API endpoints using Postman.

Software Requirements

- Node.js
- JSON Server
- Postman
- Visual Studio Code
- Command Prompt

Step 1: Verify Node.js Installation

Open Command Prompt and execute the following command:

```
node -v
```

If Node.js is not installed, download and install it. Verify the installation again using the above command.

```
C:\Users\pakal>node -v  
v22.18.0
```

Step 2: Create Project Folder

Create a folder named EmployeeRestAPI. Open Command Prompt and navigate to the project folder using: **cd <path-to-EmployeeRestAPI>**

```
[>cd C:\Users\pakal\Neha\docs\coforge\coforge_training\EmployeeRestAPI
```

Step 3: Install JSON Server

Execute the following command: **npm install -g json-server**

```
C:\Users\pakal\Neha\docs\coforge\coforge_training\StudentRestAPI>cd C:\Users\pakal\Neha\docs\coforge\coforge_training\EmployeeRestAPI  
C:\Users\pakal\Neha\docs\coforge\coforge_training\EmployeeRestAPI>npm install -g json-server  
changed 43 packages in 13s  
15 packages are looking for funding  
  run 'npm fund' for details  
C:\Users\pakal\Neha\docs\coforge\coforge_training\EmployeeRestAPI>
```

Verify the installation: `json-server --version`

```
C:\Users\pakal\Neha\docs\coforge\coforge_training\EmployeeRestAPI>json-server --version
1.0.0-beta.15
```

Step 4: Create db.json

Create a file named db.json inside the project folder.

Add the employee records into the file and save it.

Name	Date modified	Type	Size
db	11-07-2026 12:15	JSON Source File	1 KB

```
File Edit View
{
  "employees": [
    {
      "employeeId": 101,
      "firstName": "Neha",
      "lastName": "Pakala",
      "email": "nehapakala@gmail.com",
      "mobileNumber": "9876543210"
    },
    {
      "employeeId": 102,
      "firstName": "Harshika",
      "lastName": "Daval",
      "email": "harshikadaval@gmail.com",
      "mobileNumber": "9876543211"
    },
    {
      "employeeId": 103,
      "firstName": "Satvika",
      "lastName": "Harivanam",
      "email": "satvikaharivanam@gmail.com",
      "mobileNumber": "9876543212"
    },
    {
      "employeeId": 104,
      "firstName": "Riya",
      "lastName": "Gupta",
      "email": "riyagupta@gmail.com",
      "mobileNumber": "9876543213"
    },
    {
      "employeeId": 105,
      "firstName": "Ananya",
      "lastName": "Singh",
      "email": "ananyasingh@gmail.com",
      "mobileNumber": "9876543214"
    }
  ]
}
```

Step 5: Run JSON Server

Execute the following command: `json-server --watch db.json --port 3000`

The server starts successfully and displays the endpoint.

<http://localhost:3000/employees>

```
C:\Users\pakal\Neha\docs\coforge\coforge_training\EmployeeRestAPI>json-server --watch db.json --port 3000
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

( ˘ ³ ˘ )

Index:
http://localhost:3000/

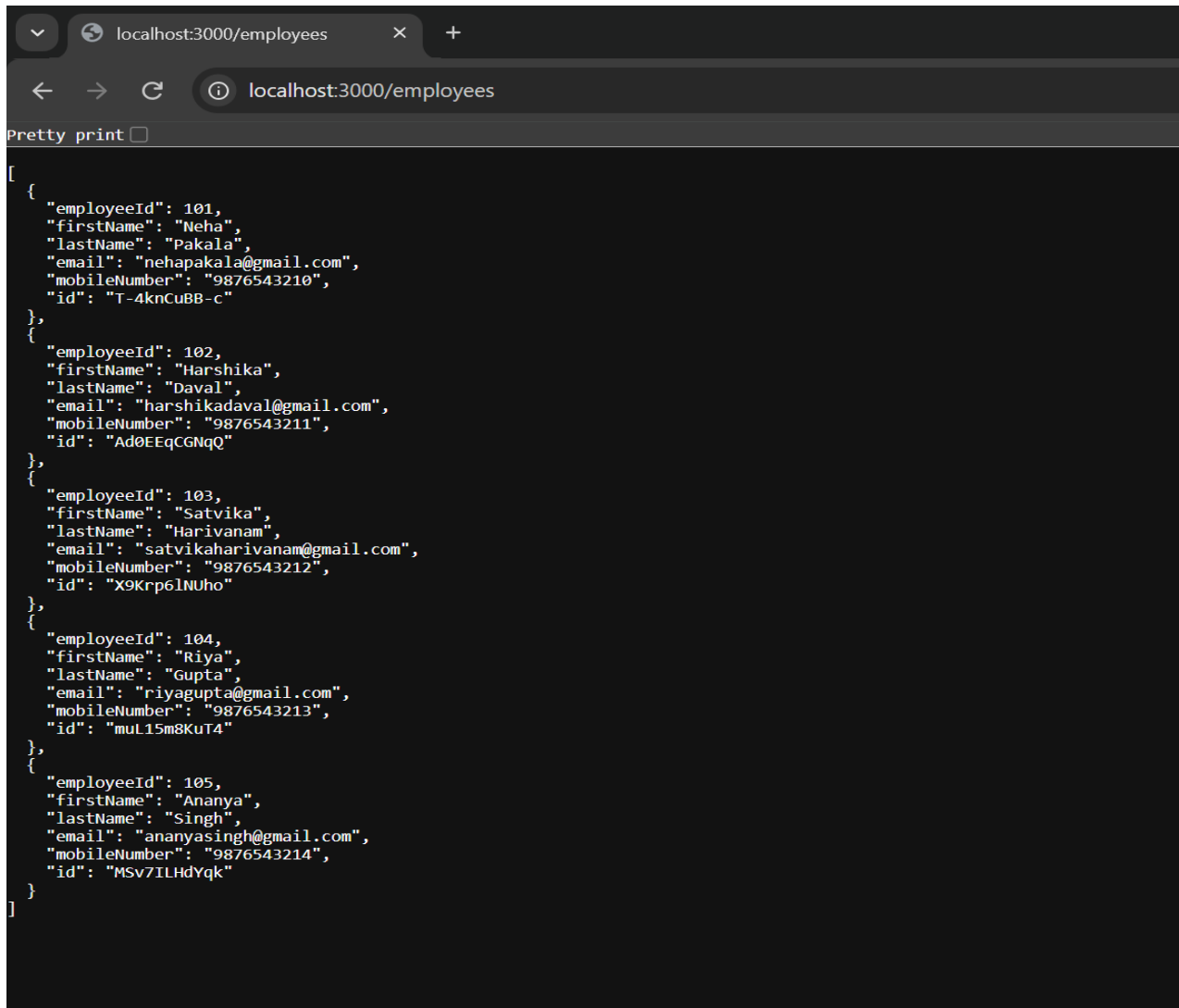
Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/employees
```

Step 6: Verify API in Browser

Open the endpoint in a web browser.

The employee records are displayed in JSON format.



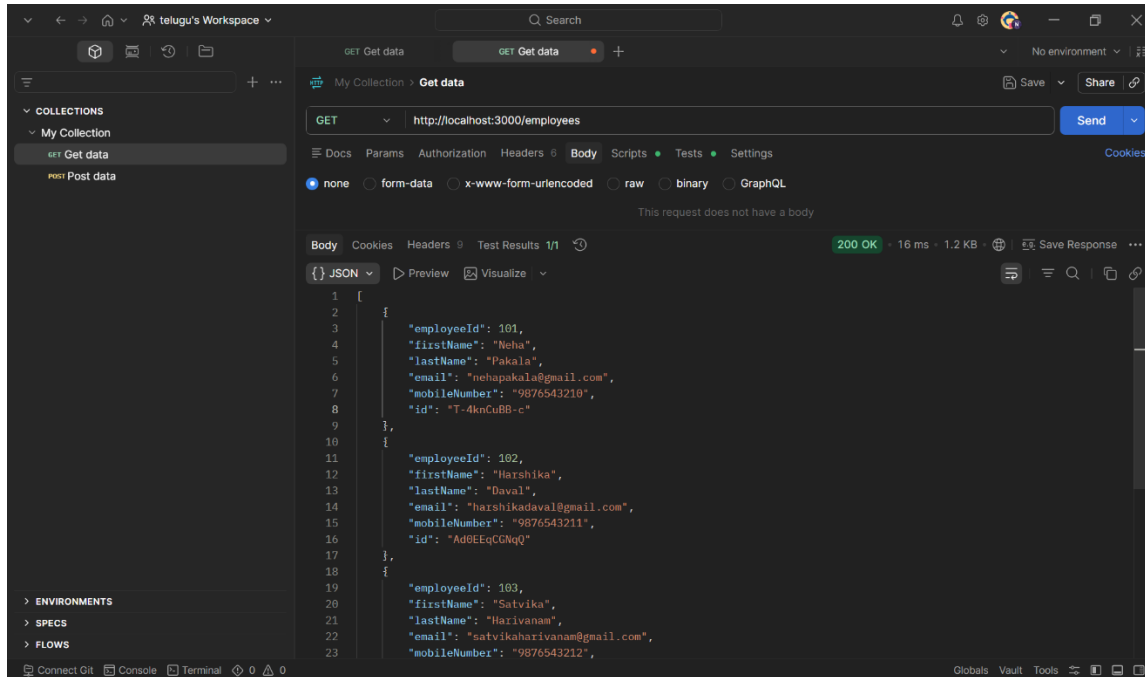
```
[
  {
    "employeeId": 101,
    "firstName": "Neha",
    "lastName": "Pakala",
    "email": "nehapakala@gmail.com",
    "mobileNumber": "9876543210",
    "id": "T-4knCuBB-c"
  },
  {
    "employeeId": 102,
    "firstName": "Harshika",
    "lastName": "Daval",
    "email": "harshikadaval@gmail.com",
    "mobileNumber": "9876543211",
    "id": "Ad0EEqCGNqQ"
  },
  {
    "employeeId": 103,
    "firstName": "Satvika",
    "lastName": "Harivanam",
    "email": "satvikaharivanam@gmail.com",
    "mobileNumber": "9876543212",
    "id": "X9Krp6lNUho"
  },
  {
    "employeeId": 104,
    "firstName": "Riya",
    "lastName": "Gupta",
    "email": "riyagupta@gmail.com",
    "mobileNumber": "9876543213",
    "id": "mul15m8KuT4"
  },
  {
    "employeeId": 105,
    "firstName": "Ananya",
    "lastName": "Singh",
    "email": "ananyasingh@gmail.com",
    "mobileNumber": "9876543214",
    "id": "MSv7ILHdYqk"
  }
]
```

Step 7: GET Method – Retrieve All Employees

Open Postman. Select the GET method.

Enter the URL: <http://localhost:3000/employees>

Click Send. The complete list of employees is displayed.

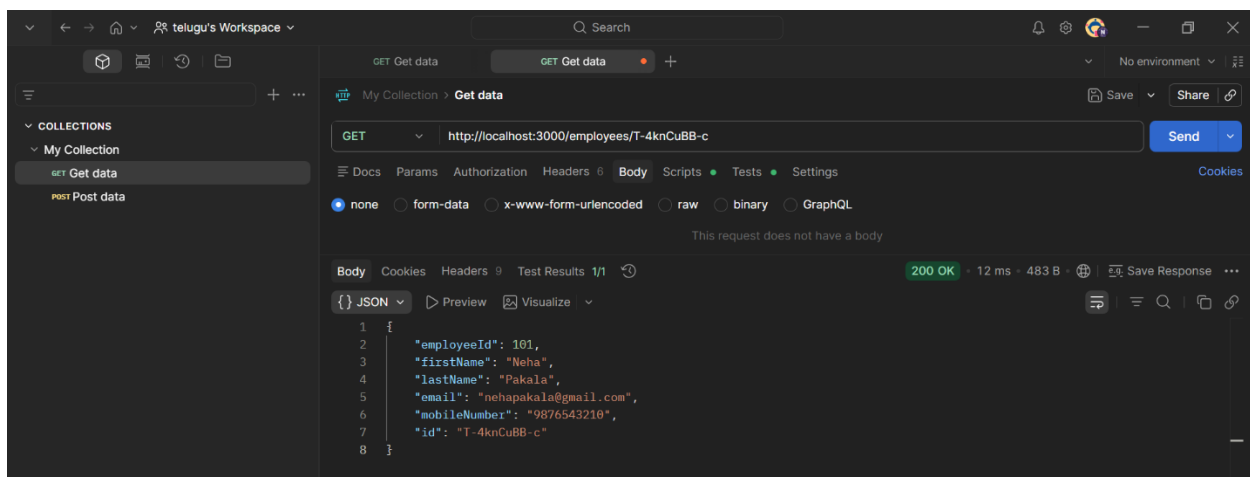


Step 8: GET Method – Retrieve Employee by ID

Select the GET method.

Enter: <http://localhost:3000/employees/T-4knCuBB-c>

Click Send. The selected employee details are displayed.

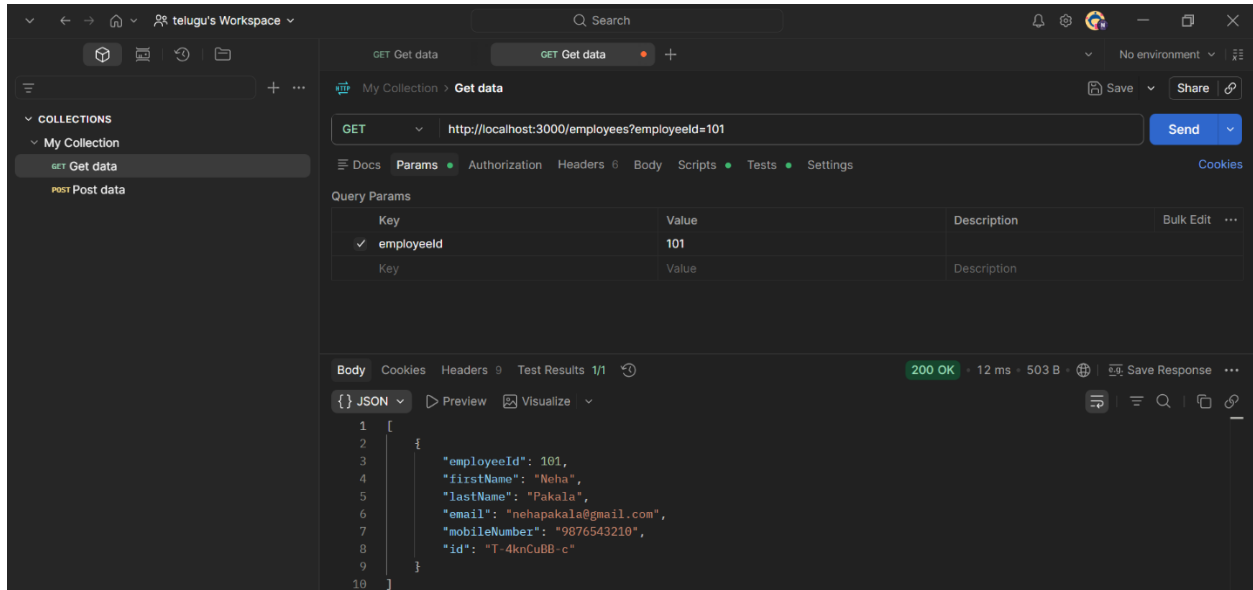


Step 9: GET Using Query Parameters

Retrieve records using a non-primary field.

Example: <http://localhost:3000/employees?employeeId=101>

Click Send. The matching employee record is displayed.

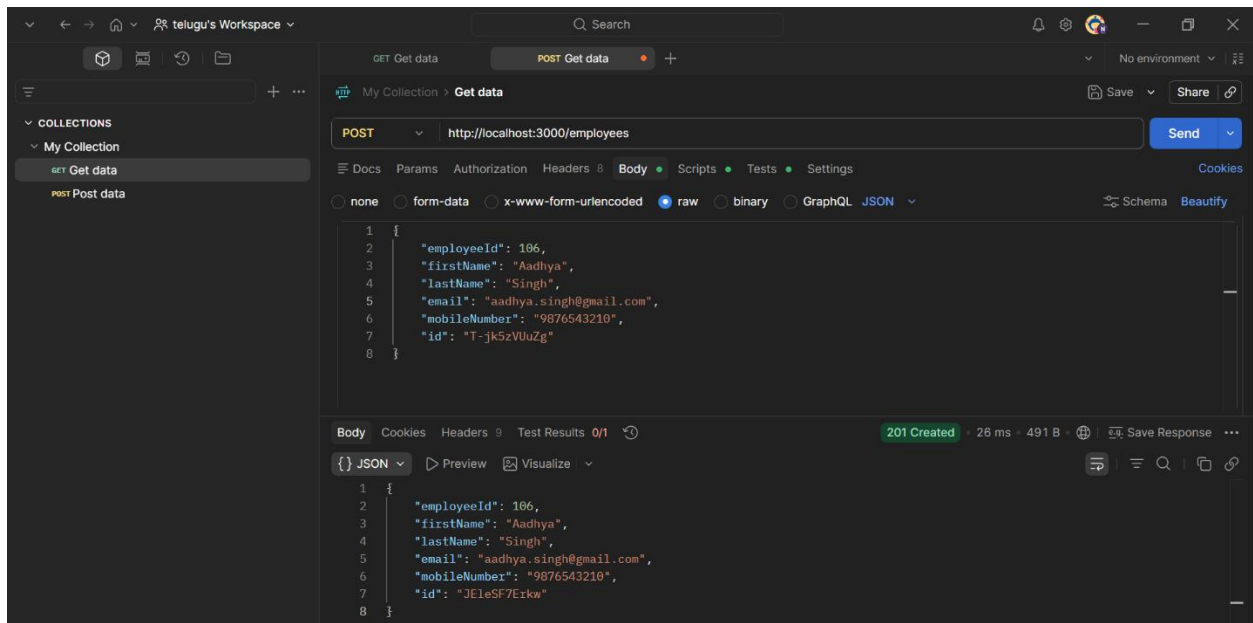


Step 10: POST Method

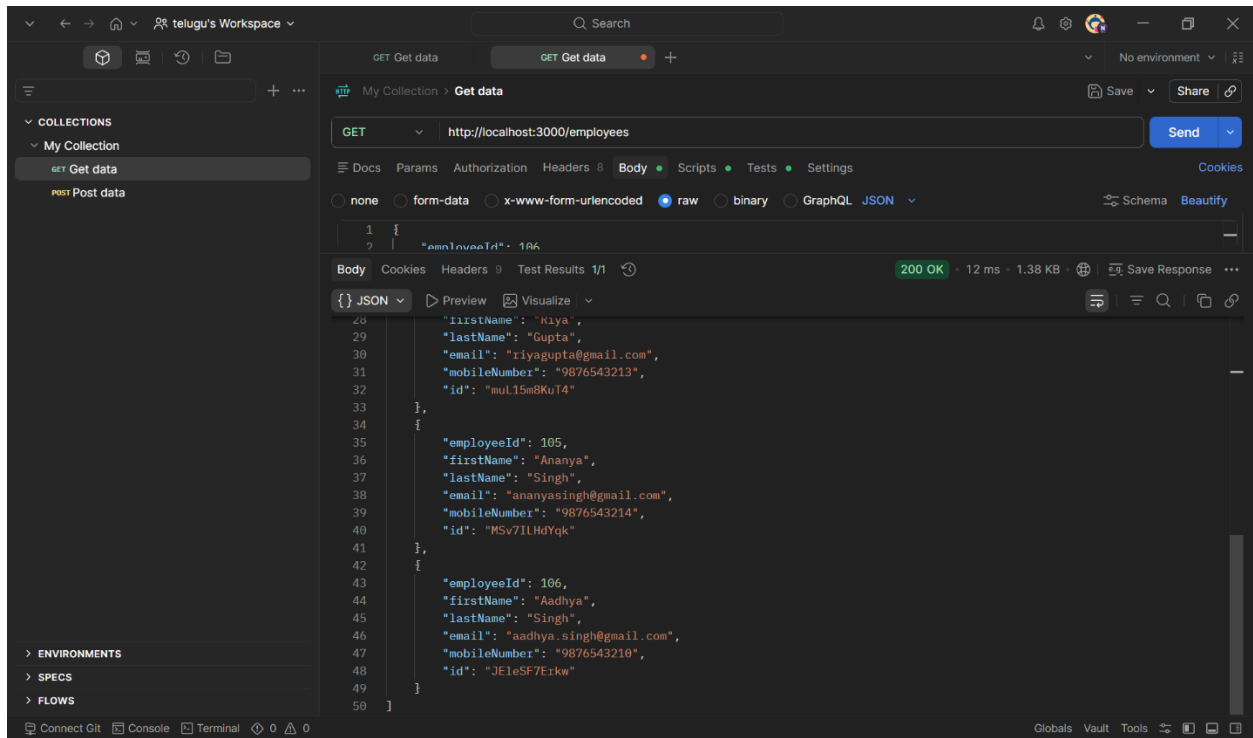
Select POST.

Enter: <http://localhost:3000/employees>

Choose Body → Raw → JSON. Paste the new employee record and click Send.



Verify that the new record has been added.

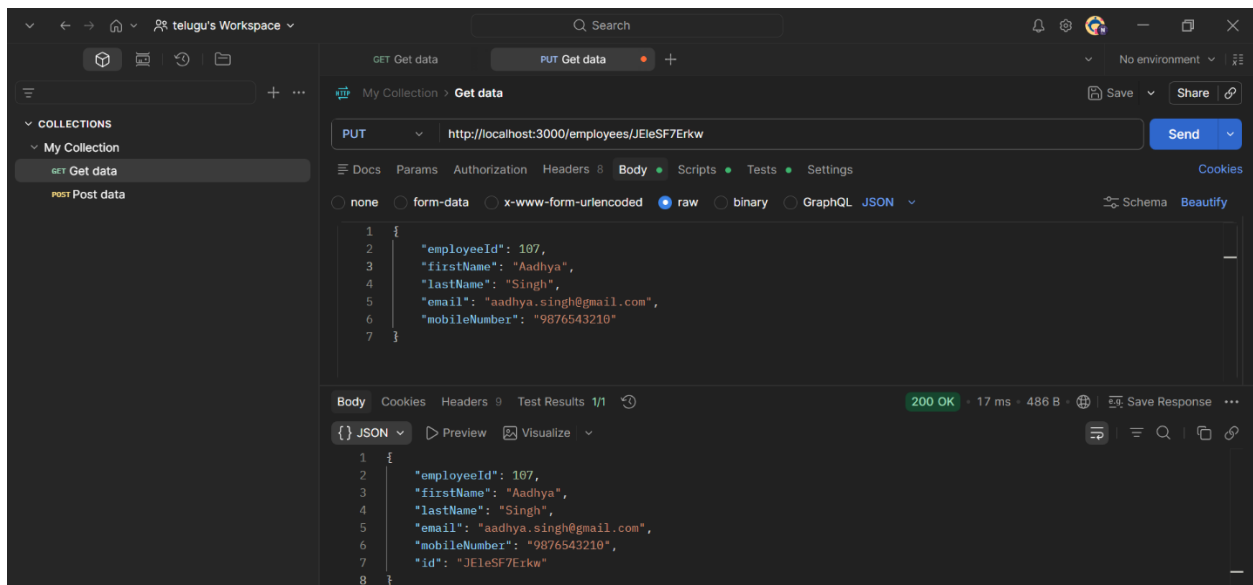


Step 11: PUT Method

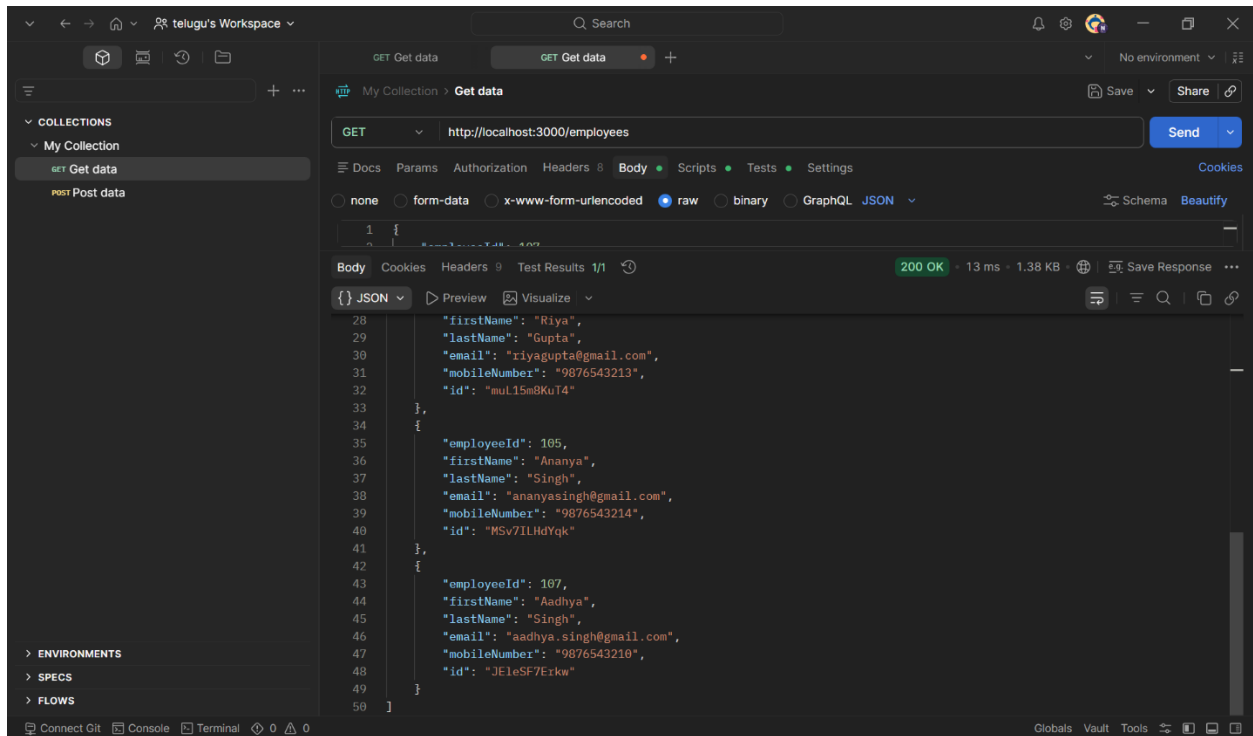
Select PUT.

Enter: <http://localhost:3000/employees/JEleSF7Erkw>

Update the employee details in the request body and click Send.



Verify the changes using a GET request.

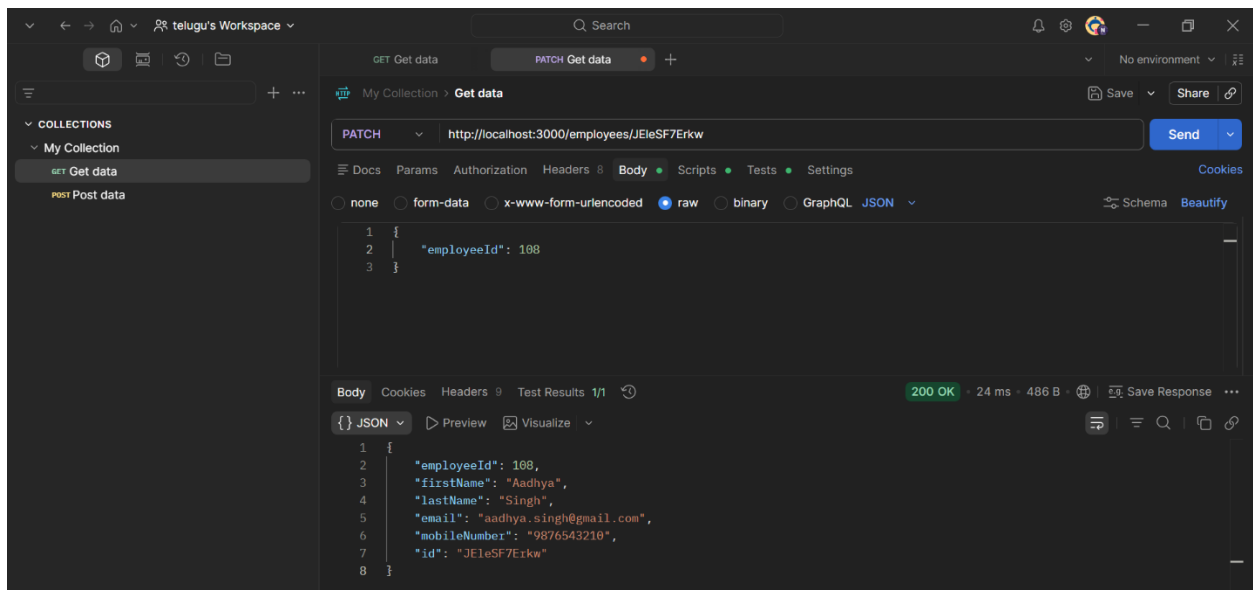


Step 12: PATCH Method

Select PATCH.

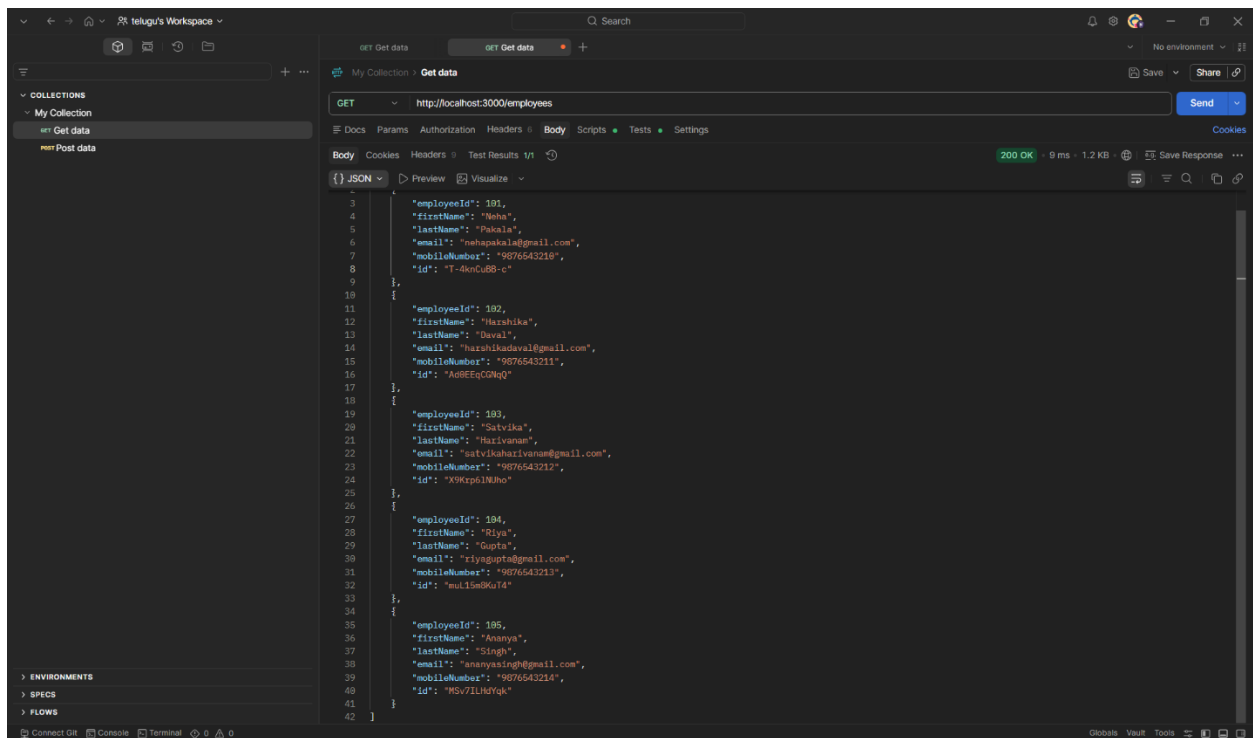
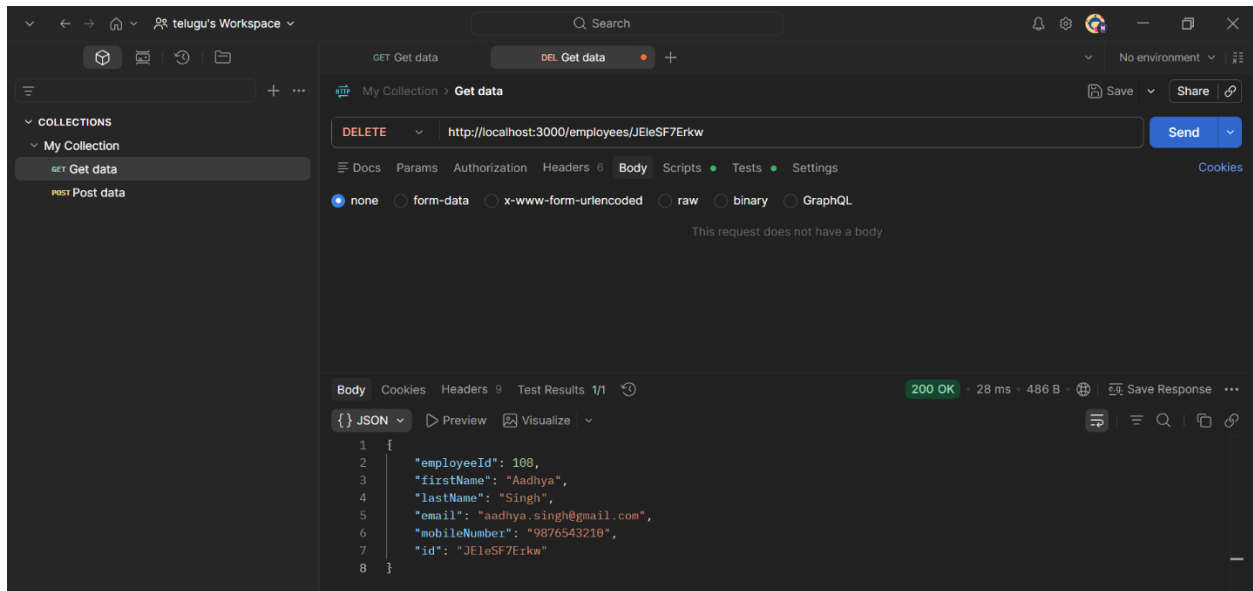
Enter: <http://localhost:3000/employees/JELeSF7Erkw>

Update only the required field and click Send. Verify the updated field using a GET request.



Step 13: DELETE Method

Select DELETE. Enter: <http://localhost:3000/employees/JEleSF7Erkw> Click Send. Verify that the employee record has been deleted.



Conclusion

Successfully configured JSON Server, created a Fake REST API using db.json, and tested all REST API endpoints (GET, POST, PUT, PATCH, and DELETE) using Postman. All CRUD operations were performed successfully.